

Problem Specification

For the final project, we decided to create the Outfit Selector. This project tackles a problem most people face every morning: figuring out what to wear. Getting dressed can require a lot of mental energy given time pressures, the desire to look put-together, and indecisiveness. Our solution to this problem is an interactive, chatbot-style outfit selector, where a user describes what they need in plain language (ex. “I want to wear something casual for class today”) and the algorithm searches through their wardrobe for an outfit suggestion. This algorithm also tracks how often each item gets worn, flags clothes as dirty after a set number of wears (also allows users to classify an item as unavailable) , and lets users see which pieces they normally reach for (favorite pieces). The project will be primarily interactive, with small data analysis components (ex. If we decide the algorithm will factor weather into its suggestion, we will need to find extensive weather data and/or utilize a weather API).

Anticipated features:

- Outfit recommendation based on texture, color, style, and weather
- Laundry hamper tracking times worn and auto-removing after a set wear count
- Least-worn and most-worn counts to identify favorite/least favorite pieces
- Live weather lookup using weather API (vs user-inputted weather data)

Needfinding Results

The needfinding interview process yielded a few consistent themes. While all interviewees noted struggling to figure out what clothing items “go together,” most placed emphasis on forgetting certain items existed in their closets (leaving limited options for outfit selection). Additionally, among the clothes that they remembered, interviewees noted frustration at pre-selected outfits becoming unviable because specific clothing items were dirty. These findings directly shaped the additional features we have decided to implement. We shifted our focus from producing cohesive, fashionable outfits to creating a comprehensive algorithm that not only chooses an outfit, but also keeps track of what you own, what needs to be washed, and how often each clothing item actually gets worn. The interviews also confirmed that the interface needs to be as low-effort as possible, driving our decision to use chatbot-style input rather than building an app or an outfit “menu.”

Grading Contract

- For a C (to pass the final project):
 - Completed PDF write up including needfinding results and algorithmic & affordance analysis
 - Valid dataset file with 10-999 entries
 - Github Repo with a README on how to run the code

- Data structure: HashMap keyed by clothing category storing ArrayLists of ClothingItem objects
- Feature: outfit recommendation where the user inputs a style and weather preference, and the system filters the wardrobe and returns a suggested outfit
- For a B:
 - What is required for a C, and
 - Valid dataset file with ≥ 1000 entries
 - Github Repo that details the custom data structure's public API
 - Feature: laundry hamper that tracks dirty clothing (items become dirty after a certain number of wears) and automatically excludes dirty items from future outfit recommendations
- For an A:
 - What is required for a B, and
 - Github Repo that details usage examples
 - Your code has good style, JavaDocs, and comments
 - Feature: leastWorn and mostWorn counts that displays the clothing items least and most worn by the user (uses a sorted list)
- For an A+
 - What is required for an A, and
 - Feature 4: using a weather API to have live weather integration where the algorithm automatically considers the current weather conditions (instead of the user typing them in)

Part 0 Slides:  Project Idea: Outfit Selector

Thoughts/Worries/Feedback

- Should the laundry hamper return items to the wardrobe after a certain period (short vs long-term use of the chatbot application)? If yes:
 - Should it automatically return items to the wardrobe after a set number of wears?
 - Should it prompt the user to confirm an item is clean before making it available again for outfit recommendations?
- Is a HashMap the better data structure (for filtering across multiple attributes at once), or is a graph better (for capturing the relationships between clothing items)
- How specific should compatibility rules be? We want to make sure we aren't overcomplicating the filtering logic in a way that sets us behind
- Does our project scope seem achievable in the amount of time we have or should we simplify anything?

- What parameters should we set around our Chatbot to detect what clothing to pick based on a given input (How does it decipher a pink blouse or blue trousers if all the user said is “formal”)? How should our algorithm work for data sorting work?